501: Linux Operating System [2511000905044001]

Unit 1: Introduction to Linux Operating System

1.1 Features of Linux OS
1.2 Components of Linux OS (Hardware, Kernel, Shell, GNU Utilities & Applications)
1.3 Shell in Linux (Bash, Zsh, Dash – Features and Differences)
1.4 Introduction to Files and File Types in Linux (text, binary, special files)
1.5 Linux Directory Structure and File System Hierarchy Standard (FHS)

What is an Operating System (OS)?

An OS manages hardware and software resources on a computer. It lets you run applications, manage files, and connect devices like printers or networks.

What is Linux ?

Linux is an open-source operating system (OS) based on **Unix**. It's widely used on servers, desktops, smartphones (like Android), embedded systems, and even supercomputers.

Key Features of Linux:

1. **Open-source**: The source code is freely available for anyone to view, modify, and distribute.
2. **Free to use**: Most Linux distributions (called "distros") are free to download and use.
3. **Secure and Stable**: Known for its stability and strong security features.
4. **Multiuser and Multitasking**: Multiple users can use the system at once, and it can run many tasks simultaneously.
5. **Customizable**: You can change nearly every part of the system.

Popular Linux Distributions (Distros):

- **Ubuntu** – user-friendly, great for beginners
- **Debian** – stable and widely used for servers
- **Fedora** – cutting-edge features, backed by Red Hat
- **CentOS / AlmaLinux / Rocky Linux** – used in enterprise environments
- **Arch Linux** – for advanced users who want full control

Where is Linux Used?

- Web servers (e.g., Apache, NGINX on Linux)
- Supercomputers (most top supercomputers run Linux)

- Smartphones (Android is based on Linux)
- IoT and embedded systems
- Developers' laptops and desktops

Components of Linux OS ?

Linux, like other operating systems, is made up of several key components that work together to manage the computer's hardware and software.

Linux OS Structure (Simplified Diagram):

Applications
Shell
System Utilities
System Libraries
Kernel
Hardware

1. Kernel

- **Core part** of the OS.
- Manages system resources (CPU, memory, devices).
- Communicates between hardware and software.
- Types: **Monolithic kernel** (Linux kernel is monolithic).

A **monolithic kernel** is an architecture where the entire operating system (OS) runs as a single large process in **kernel space**. It includes the **core functions** such as:

- Process management
- Memory management
- Device drivers
- File system
- System calls
- Interrupt handling

All these components run together in **one large block of code**.

Example: Memory management, process scheduling, device drivers.

2. System Library

- Special functions or programs used by applications to interact with the kernel.
- Avoids the need for apps to directly interact with the kernel.

Example: GNU C Library (glibc) helps software use system calls.

The GNU C Library, commonly known as glibc, is the standard C library used in GNU/Linux systems.
It provides the core libraries for the C programming language, including:

- System calls (e.g., open, read, write)
- Memory management (e.g., malloc, free)
- Process control (e.g., fork, exec)
- String handling (e.g., strcpy, strlen)
- File operations

3. System Utility

- Basic tools and commands for managing the system.
- Help users and admins perform tasks like copying files, managing processes, and configuring networks.

Examples: cp, mv, top, ifconfig, ps

Command	Purpose	Example
cp	Copy files or folders	cp file.txt /backup/
mv	Move/rename files or folders	mv a.txt b.txt
top	Monitor system in real time	top
ifconfig	Show network config (IP, MAC)	ifconfig
ps	Show running processes	ps aux

4. Shell

- Interface between the user and the kernel.
- Accepts user commands and sends them to the kernel for execution.
- Types:
  - **CLI (Command Line Interface)** – e.g., **Bash**, **Zsh**
  - **GUI Shell** – graphical interface (e.g., GNOME Terminal)

Example: Typing ls in a terminal shows files in a directory.

5. Hardware Layer

- Physical components: CPU, RAM, hard drive, keyboard, etc.
- Linux accesses hardware via the kernel and device drivers.

6. Application Layer

- User-level programs and apps.
- These run on top of the OS and include everything from browsers to text editors.

Examples: Firefox, LibreOffice, GIMP, VS Code

What is a Shell in Linux?

A **shell** is a command-line interface (CLI) that allows users to interact with the **Linux operating system**. It interprets the commands entered by the user and communicates with the system to execute them. It acts as a **bridge between the user and the kernel**.

Functions of a Shell:

- Accepts **commands** from the user.
- Interprets and passes them to the **kernel**.
- Displays the **output** or result of those commands.
- **Types of Shells in Linux:**

Shell Name	**Description**	**Command to Use**
Bash (Bourne Again Shell)	Default in many Linux distros, user-friendly	bash
Sh (Bourne Shell)	Original Unix shell, simple	sh
Zsh (Z Shell)	Advanced shell, supports plugins	zsh
Ksh (Korn Shell)	Combines features of sh and csh	ksh
Tcsh (TENEX C Shell)	C-like syntax, user-friendly	tcsh

There are various types of shells available in Linux. The most commonly used are:

- **Bash (Bourne Again Shell)**
- **Zsh (Z Shell)**
- **Dash (Debian Almquist Shell)**

1. **Bash (Bourne Again Shell)**

Features:

- Default shell on most Linux distributions.
- Supports **command history**, **auto-completion**, and **scripting**.
- Allows use of variables, functions, loops, and conditionals.
- Strong **backward compatibility** with the original Bourne shell (sh).
- Good support for **interactive** and **non-interactive** usage.

Example:

```
bash
echo "Welcome to Bash Shell"
```

2. **Zsh (Z Shell)**

Features:

- More **customizable and feature-rich** than Bash.
- **Auto-suggestions** based on command history.
- **Improved tab completion** with menu selection.
- Supports **themes and plugins** (Oh-My-Zsh is a popular framework).
- Has built-in **spell-checking**, **path expansion**, and **better scripting features**.

Example:

```
zsh
echo "Hello from Zsh Shell"
```

3. **Dash (Debian Almquist Shell)**

Features:

- **Lightweight and fast**, optimized for **shell scripting**.
- Used by Ubuntu and Debian as the default /bin/sh for system scripts.
- Not meant for interactive use (lacks user-friendly features).
- Executes scripts **faster than Bash**, useful for boot scripts and system-level scripting.

Limitations:

- Does **not support** many Bash-specific features (like arrays or [[...]] test syntax).

Example:

```
dash
echo "This script runs faster with Dash"
```

Comparison Table:

Feature	Bash	Zsh	Dash
Default Shell	Most Linux distros	Optional	Ubuntu/Debian /bin/sh
Interactive Features	Yes	Advanced	Minimal
Auto-completion	Yes	Better with plugins	No
Scripting Performance	Moderate	Moderate	High (fast)
Plugin/Themes Support	Limited	Extensive (via Oh-My-Zsh)	No
Compatibility with sh	High	Medium	Very High
Used for System Scripts	Sometimes	Rarely	Often

Note :

- **Bash** is user-friendly and widely used, perfect for beginners and general use.
- **Zsh** is for advanced users who want more productivity and customization.
- **Dash** is for fast, efficient script execution in system-level operations.

4. **Bourne Shell**

The **Bourne Shell**, known as sh, was developed by **Stephen Bourne** in **1979** at **AT&T Bell Labs**.

It was the **first standard shell** in Unix and became the base for many other shells like **Bash, Dash, Ksh**.

It is used to run shell scripts and execute system commands.

Features of sh (Bourne Shell)

- Portability: Scripts written in sh can run on almost all Unix and Linux systems.
- Basic Scripting Support: Allows variables, loops, conditionals, and functions.
- Control Structures: Supports if, else, elif, while, for, and case statements.
- Input/Output Redirection: Handles >, <, >>, and | for directing input/output.
- Command Substitution: Uses backticks `command` to run commands within commands.
- Simple Syntax: Easy to write and understand basic scripts.
- Efficient Execution: Lightweight and fast; suitable for system-level tasks.
- Script Compatibility: Can be used as /bin/sh in many Linux distributions.
- Minimal Environment: Doesn't include advanced features but is reliable for basic scripting.
- Used in System Scripts: Commonly used in boot and initialization scripts (e.g., /etc/init.d/).

Korn Shell (ksh)

- Developed by **David Korn** at Bell Labs in the early 1980s.
- A **superset of the Bourne Shell (sh)** with features from C Shell (csh) and additional programming capabilities.

- Compatible with scripts written for **Bourne Shell**.

Features:

1. **Command history** – Recall and reuse previous commands using history.
2. **Job control** – Suspend, resume, or move processes to background.
3. **Command aliasing** – Shortcuts for long commands.
4. **Built-in arithmetic** – Supports integer arithmetic using (( )).
5. **Script control structures** – if, for, while, select, case, etc.
6. **Arrays support** – One of the first shells to support arrays.
7. **Functions and local variables** – Allows modular programming.
8. **Command-line editing** – Emacs or vi-style keyboard shortcuts.
9. **Enhanced I/O redirection** – >|, <>, and more advanced usage.
10. **Faster execution** – Optimized for performance compared to sh or csh.

Note:

- **Korn Shell (ksh)** is a **powerful scripting shell** suitable for programming and automation.
- It offers **robust scripting features** and is used in **enterprise-level UNIX/Linux systems**.

TENEX C Shell (tcsh)

- An enhanced version of **C Shell (csh)** developed at the **Carnegie Mellon University**.
- tcsh is backward-compatible with csh and includes improvements from the **TENEX** operating system.

Features:

1. **C-like syntax** – Familiar for programmers from C language background.
2. **Command completion** – Auto-completes filenames and commands with TAB.
3. **Command history** – Browse and edit previous commands.
4. **Spelling correction** – Auto-fix typos in commands and filenames.
5. **Job control** – Background, foreground, suspend jobs using fg, bg, jobs.
6. **Alias support** – Define command shortcuts.
7. **User-friendly prompt customization** – Can display current directory, hostname, etc.
8. **Enhanced editing** – Line editing features and command-line shortcuts.
9. **Wildcards and globbing** – Like *, ?, [] for pattern matching.

Note:

- **TENEX C Shell (tcsh)** is more **user-friendly for interactive use**, especially with **command-line editing**, **auto-completion**, and **C-like syntax**, but is **less preferred for complex scripting**.

Files and File Types in Linux:

In Linux, **everything is treated as a file**—including directories, hardware devices, and even processes.

Files in Linux are the primary means of storing and accessing data.

Unlike Windows, Linux doesn't rely on file extensions to identify file types but instead uses **metadata and content** to determine the type of a file.

Linux File system Structure :

- Files are organized in a hierarchical directory structure (rooted at /).
- Each file has **permissions, ownership, timestamps**, and a **type**.

What Are Files in Linux?

In Linux, **everything is a file** — not just documents or pictures, but also folders, programs, and even devices like your keyboard or hard drive.

Types of Files in Linux

Linux has different types of files. The most common are:

1. Text Files

- These files contain **readable text**.
- You can open and edit them using text editors like nano, vim, or gedit.

Examples:

- Notes or documents: notes.txt
- Code files: program.py, script.sh
- System settings: /etc/hostname, /etc/passwd

Example Command:

bash

cat file.txt # Show the content of a text file

2. Binary Files

- These files contain **computer-readable data**, not meant for humans.
- They are used by programs or the system.
- You can't read them properly with a text editor — they look like random symbols.

Examples:

- Program files: /bin/ls, /usr/bin/python3
- Images or videos: image.jpg, video.mp4

Example Command:

bash

file /bin/ls # Tells you it is a binary (executable) file

3. Special Files

These are special types of files that help the system **talk to devices** or **allow programs to talk to each other**.

There are a few types of special files:

a) Device Files

- Found in /dev/ folder.
- Used to connect with hardware like USB drives, hard disks, etc.
- Two kinds:
  - **Character devices** – transfer data one character at a time (e.g., keyboard).
  - **Block devices** – transfer data in chunks (e.g., hard drive).

Example:

bash

/dev/sda1 # a block device (like a hard drive partition)

b) Named Pipes (FIFOs)

- Help programs send data to each other.
- Think of it like a one-way tube: one program sends, the other receives.

bash

mkfifo mypipe # creates a pipe file

c) Symbolic Links (Shortcuts)

- These are **shortcuts to other files**.
- Like a shortcut on your desktop that opens a file in another location.

bash

ln -s real_file.txt shortcut.txt

d) Sockets

- Used when programs need to **talk to each other**, especially over a network.

Example: /var/run/docker.sock (used by Docker)

How to Check File Type

Use this command to know what kind of file something is:

bash
file filename

Use this to see file types in a folder:

bash

ls -l

Look at the first letter of each line:

Symbol Meaning

Symbol Meaning

- Regular file
d Directory
l Symbolic link
c Character device
b Block device
p Pipe (FIFO)
s Socket

File Type	What It Is	Example
Text File	Readable by humans	notes.txt
Binary File	Readable by computer only	/bin/ls, image.jpg
Device File	Used to connect to hardware	/dev/sda, /dev/tty
Pipe (FIFO)	Program-to-program communication	mypipe
Socket	Communication between programs	/var/run/docker.sock
Symlink	Shortcut to another file	shortcut.txt

What Is the File System Hierarchy Standard (FHS)?

The **FHS** is a standard that defines **which directories should exist** and **what they should contain** in Linux systems. It helps keep things **organized** and **consistent**.

Common Directories

Directory	What It's For (Simple Explanation)
/	Root directory. The **top** of the filesystem. Everything starts here.
/bin	**Essential programs** used by the system and all users.
/sbin	**System-only programs**, usually for admin tasks.
/boot	Files needed to **boot** (start) the system (like the Linux kernel).
/dev	Files that represent **devices** (like hard drives, USBs).
/etc	**Configuration files** for the system and programs.
/home	Personal folders for **regular users** (e.g., /home/alex).
/lib	**Libraries** needed by programs in /bin and /sbin.
/media	Used to **automatically mount** USBs, CDs, etc.

Directory	What It's For (Simple Explanation)
/mnt	Temporary mount point (e.g., for manually mounted drives).
/opt	Optional software or **third-party apps**.
/proc	Virtual folder for **process info** and system details (not real files).
/root	Home folder for the **root (admin) user**.
/run	Temporary data for **running processes**.
/srv	Data for **services** (like web servers).
/tmp	**Temporary files**; gets cleared on reboot.
/usr	Programs and files for **users and applications** (not essential for boot).
/var	Files that **change often**, like logs, emails, print jobs, etc.

/usr Subdirectories (for user-level programs)

Subdirectory	Purpose
/usr/bin	Most user programs
/usr/sbin	System admin tools for regular use
/usr/lib	Libraries for /usr/bin and others
/usr/share	Shared data (icons, docs, etc.)
/usr/local	Programs installed by the user

Note:

- /is the **starting point** of everything.
- /home is where **your personal files** live.
- /etc holds **settings**.
- /bin, /sbin, /lib, /usr, and /var are **core system directories**.
- /dev, /proc, and /sys are **virtual** – they represent system data, not actual files on disk.

Example

If you are user jaimini, your files are in:

bash

/home/jaimini/

If you want to check system logs:

bash

/var/log/

If you want to see installed apps:

bash

/usr/bin/

Linux Directory Structure & File System Hierarchy Standard (FHS)

In Linux, the **File System Hierarchy Standard (FHS)** defines **how files and directories should be organized**. It helps all Linux distributions (like Ubuntu, Fedora, Red Hat, etc.) to follow the **same structure**, so users and software know where to find files.

Think of it like a **library system** — with specific sections for books, magazines, newspapers, and archives. FHS makes it easy to keep things organized and accessible.

The Root Directory /

Everything in Linux starts from a **single root directory** called / (a forward slash).

From /, all other directories branch out like a **tree structure**.

Important Directories in Linux (with Explanation)

Directory	Description	Example Contents
/	Root directory: the top of the filesystem	All other folders are inside it
/bin	Essential **binary commands** for all users	ls, cp, mv, rm
/sbin	**System binaries**: Commands for system admin	fsck, reboot, ifconfig
/boot	Files needed to **boot the system**	vmlinuz (Linux kernel), grub/
/dev	Represents **devices** as files	/dev/sda (hard disk), /dev/usb
/etc	System **configuration files**	/etc/hostname, /etc/fstab
/home	**User home directories**	/home/alex, /home/sara

Directory	Description	Example Contents
/lib	Libraries for programs in /bin and /sbin	libc.so, ld-linux.so
/media	Auto-mount location for **removable devices**	USBs, CDs
/mnt	Temporary mount location for **manual mounting**	Custom mounts
/opt	Optional software or **third-party apps**	/opt/google/, /opt/skype/
/proc	Virtual directory for **process info**	/proc/cpuinfo, /proc/uptime
/root	Home directory for the **root (admin) user**	Only root can access
/run	Temporary system data like **PIDs, sockets**	/run/systemd/, /run/user/
/srv	Data for **services** like web or FTP servers	/srv/www, /srv/ftp
/sys	Info about **hardware devices and drivers**	/sys/class/, /sys/block/
/tmp	Temporary files. Deleted after reboot.	App caches, temp logs
/usr	Secondary hierarchy for **user applications**	/usr/bin, /usr/lib
/var	Variable data that **changes often**	/var/log, /var/mail

Important Directories

/bin – Basic Commands

- Contains important **executable programs**.
- Available even in **single-user mode** or during **booting**.
- Examples: ls, echo, cat, cp, mv

/sbin – System Commands

- Contains system **admin tools**.
- Usually used by **root** (admin).
- Examples: reboot, fdisk, iptables

/home – User Files

- Each user has a folder here:
    - /home/john, /home/maria

- Contains documents, downloads, pictures, etc.

/etc – System Configuration

- Contains **settings** for software and the OS.
- These are mostly **text files**.
- Examples:
  - /etc/passwd: user accounts info
  - /etc/hosts: maps IPs to hostnames

/proc – Process Info

- A **virtual folder** (not on disk).
- Shows **live info** about the system and processes.
- Example:
  - /proc/uptime (system uptime)
  - /proc/1/ (info about process ID 1)

/dev – Devices as Files

- All hardware devices appear as **files**.
- Example:
  - /dev/sda = first hard drive
  - /dev/tty = terminal

/tmp – Temporary Files

- Programs store **temporary data** here.
- Auto-deleted on system reboot.

Subdirectories Inside /usr – User Programs and Libraries

Directory	**Purpose**
/usr/bin	User programs (non-essential) like firefox, nano
/usr/sbin	System programs for admin (not needed at boot)
/usr/lib	Libraries for /usr/bin and /usr/sbin
/usr/local	Software installed **by the user**, not by the OS

Subdirectories Inside /var – Variable Data

Directory	Purpose
/var/log	Log files (system and app messages)
/var/mail	User emails
/var/tmp	Temporary files that need to survive reboot
/var/cache	App cache data

Special Directories

Directory	Meaning
/root	Home folder for **root user**
/opt	Optional third-party software
/srv	Data served by the system (like a website)
/run	Info about **currently running** processes and services

Directory	What it Stores	Who Uses It
/	Root of all directories	Everyone
/bin	Basic commands	All users
/sbin	Admin tools	Root/admin
/home	Personal files	Users
/etc	Settings/configs	System
/usr	Software and tools	Users & apps
/var	Changing data (logs, mail)	System
/tmp	Temporary files	Apps
/dev	Device files	System
/proc, /sys	Virtual files (system info)	System
/boot	Boot loader & kernel	System
/root	Root's home directory	Root only